[39] X

	body_len	punct%	0	1	2	3	4	5	6	7	...	13165	13166	13167	13168	13169	13170	13171	13172	13173	13174
0	40	5.0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	37	2.7	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	41	2.4	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	102	1.0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	20	5.0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2196	324	7.4	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2197	1110	6.3	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2198	928	5.5	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2199	464	9.3	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2200	26	0.0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

2201 rows × 13177 columns

Figure 4: Sample of final features

training part. We will separate the data because we want to test on a new set of data whether our model is working properly or not.

Now let us start with the first model. We will try the KNN algorithm first, and use the *sklearn* library for this. We will first train the model and then assess its performance (all of this can be done using the *sklearn* library in Python itself). The following piece of code does this, and we get an accuracy of around 50 per cent.

```
from sklearn.neighbors import
KNeighborsClassifier
model = KNeighborsClassifier (n_
neighbors=3)
model.fit(X_train, y_train)
model.score (X_test,y_test)
```

```
0.5056689342403629
```

The code is similar in the logistic regression model — we first import the function from the library, fit the model, and then test it. The following piece of code uses the logistic regression algorithm. The output shows we got an accuracy of around 66 per cent.

```
from sklearn.linear_model import
LogisticRegression
model = LogisticRegression()
model.fit (X_train,y_train)
model.score (X_test,y_test)
```

```
0.6621315192743764
```

The following piece of code uses SVM. The output shows we got an accuracy of around 67 per cent.

```
from sklearn import svm
model = svm.SVC(kernel='linear')
model.fit(X_train, y_train)
model.score(X_test,y_test)
```

```
0.6780045351473923
```

The following piece of code uses the Random Forest algorithm, and we get an accuracy of around 69 per cent.

```
from sklearn.ensemble import
RandomForestClassifier
model = RandomForestClassifier()
model.fit(X_train, y_train)
model.score(X_test,y_test)
```

```
0.6938775510204082
```

Next we use the Decision tree algorithm, which gives an accuracy of around 61 per cent.

```
from sklearn.tree import
DecisionTreeClassifier
model = DecisionTreeClassifier()
```

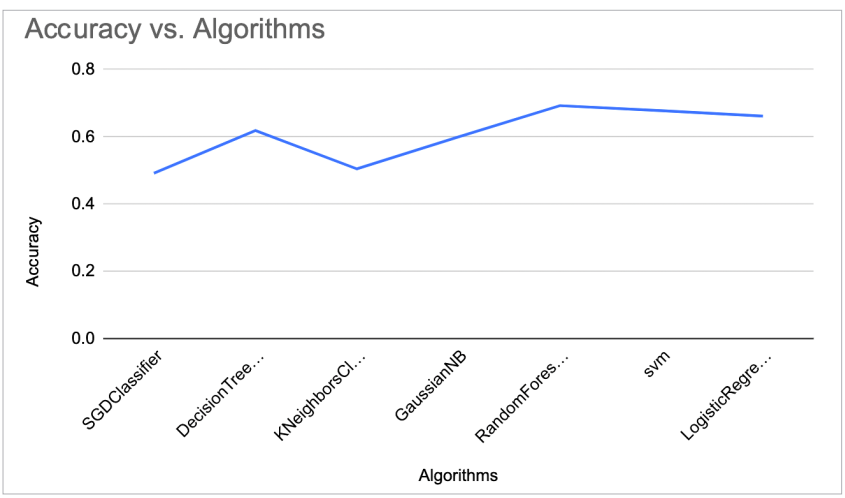

Figure 5: Accuracy performance of the different algorithms